DIGITAL IMAGE COMPRESSION STANDARDS

Lossless and lossy compression

- Compression is the means to have an encoded representation of the image that provides the same (to some extent) appearance with a lower number of bits. Compression can be lossless or lossy. What compression technique is to be used depends on the application. In general:

  - Text documents: lossless compression
  - Data for numerical analysis: lossless compression
  - Programs: lossless compression
  - Typographic images: lossless compression
  - WEB images: lossy compression
  - Video: lossy compression
  - Audio: lossy compression

- Lossy compression guarantees higher compression rates

Lossless compression

Run-lenght coding

- *Run lenght coding* is a fixed-lenght coding scheme. With run lenght encoding, a sequence of equal symbols is encoded with only one symbol, preceded by a special symbol (e.g $) followed by the number that specifies the times the symbol appears consecutively

- For example, consider a text string composed of 3 sequences, each one composed of of 11 “r”, followed by 11 “p” and then 11 “c”:

  rrrrrrrrrrrpppppppppppccccccccccc
  rrrrrrrrrrrpppppppppppccccccccccc
  rrrrrrrrrrrpppppppppppccccccccccc

  total dimension of 99 bytes.

- The sequences are encoded as:

  $11r $11p $11c
  $11r $11p $11c
  $11r $11p $11c

  i.e.:

  Total characters ($, r, p, c): 6+6+6 = 18
  Total numbers (11): 3+3+3=9

  Resulting in a total dimension of 27 Byte = 216 bit

Was used in PCX image format.
Nowadays is used as a part of a more complex compression algorithm

Run-lenght coding: a simple Python example

```
escape_char = "$"

def encode_rle(string):
    encoded = ""
    count = 1
    prev_char = string[0]

    for char in string[1:]:
        if char == prev_char:
            count += 1
        else:
            if count > 3:
                encoded += escape_char + "{}{}".format(count, prev_char)
            else:
                encoded += prev_char * count
            count = 1
            prev_char = char

    # Handle the last character sequence
    if count > 3:
        encoded += escape_char + "{}{}".format(count, prev_char)
    else:
        encoded += prev_char * count

    return encoded
```

Input string:
aaaaahhbbbbfcccccdddddddeeeggggggggggg
Encoded string: $5ahh$4bf$5c$6deee$11g

Run-lenght coding: a simple Python example

```
escape_char = "$"

def decode_rle(encoded_string):
    decoded = ""
    i = 0

    while i < len(encoded_string):
        if encoded_string[i] == escape_char:
            j = i + 1
            count_str = ""
            while j < len(encoded_string) and encoded_string[j].isdigit():
                count_str += encoded_string[j]
                j += 1
            count = int(count_str)
            char = encoded_string[j]
            decoded += char * count
            i = j + 1  # Move the index past the encoded part
        else:
            decoded += encoded_string[i]
            i += 1

    return decoded
```

Input string: $5ahh$4bf$5c$6deee$11g
Decoded string:
aaaaahhbbbbfcccccddddddeeeggggggggggg

Run-lenght coding: compact symbol representation

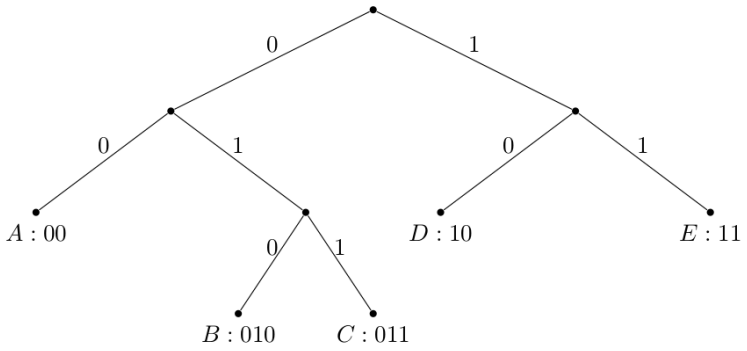
- There are several efficient ways to represent RLE symbols compactly:
- Reserve specific bit patterns for counts:
  - Use the most significant bit to indicate if a byte is a count (1) or literal (0)
- Utilize variable-length codes for counts:
  - Elias gamma coding: especially efficient for small counts
  - Delta coding: better for larger numbers than gamma
- PackBits Algorithm
  - Negative count byte: repeat the next byte n times
  - Positive count byte: copy the next n+1 bytes literally
- Combine RLE with other compact coding like Huffman, LZ77, etc.

Prefix-free coding

- Fixed-length codes although always uniquely decipherable do not always give the best compression. To improve compression variable length codes are preferred.

- *Prefix free coding* is a coding scheme where each symbol is represented by a codeword and the codeword is never a prefix of the bit string representing any other symbol
- Every message encoded by a prefix free code is uniquely decipherable. Since no codeword is a prefix of any other symbol we can always find the first codeword in a message, peel it off, and continue decoding.

E.g.: 1011011000100010
A: 00
B: 010
C: 011
D: 10
E: 11



Prefix free coding: a simple Python example

```
class PrefixFreeCode:
    def __init__(self, codebook):
        self.codebook = codebook  # a dict {K : v, ...}
        self.reverse_codebook = {v: k for k, v in codebook.items()}

    def encode(self, message):
        encoded_message = ""
        for char in message:
            encoded_message += self.codebook[char]
        return encoded_message

    def decode(self, encoded_message):
        decoded_message = ""
        current_code = ""
        for bit in encoded_message:
            current_code += bit
            if current_code in self.reverse_codebook:
                decoded_message += self.reverse_codebook[current_code]
                current_code = ""
        return decoded_message
```

Huffman coding

- *Huffman coding* refers to the use of a variable-length, prefix-free code for encoding a source symbol.

- With Huffman coding:
  - a prefix code is used for each symbol that expresses how much frequently the symbol is used: the shorter code the more common symbol is
  - the bit string representing some particular symbol is never a prefix of the bit string representing any other symbol (prefix-free code)

- It is the most efficient compression method of this type: no other mapping of individual source symbols to unique strings of bits will produce a smaller average output size with the same symbol frequencies.

- The Huffman code for a set of symbols may be generated by constructing a binary tree with nodes containing the symbols to be encoded and their frequencies of occurrence. The tree may be constructed as follows:
  - Step 1. Select the two parentless nodes with the lowest frequencies.
  - Step 2. Create a new node which is the parent of the two lowest frequency nodes.
  - Step 3. Assign the new node a frequency equal to the sum of its children's frequencies.
  - Step 4. Repeat Step 1 until there is only one parentless node left.

- The code for each symbol may be obtained by tracing a path to the symbol from the root of the tree: **1** is assigned for a branch in one direction and **0** is assigned for a branch in the other direction.

- The running time of Huffman's method is fairly efficient, it takes O (n log n) operations to construct it.

- Huffman code Example

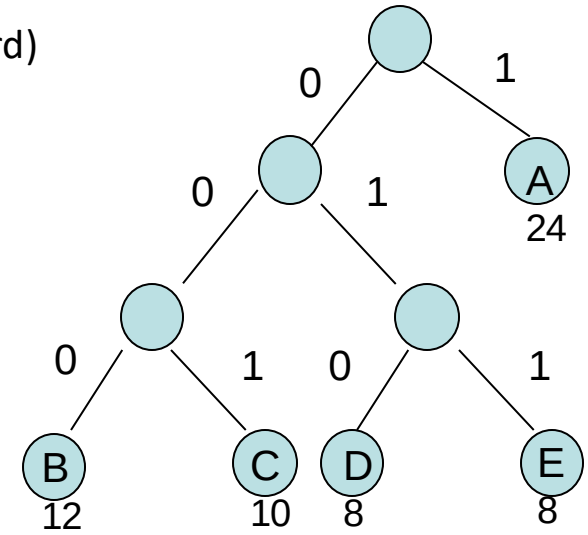
Consider a set of five different symbols. The symbol's frequencies are:

A	24
B	12
C	10
D	8
E	8

If not encoded, this results into a total of 186 bit (3 bit per codeword)

Huffman encoding:

- Step 1. Combine D and E into DE with a frequency of 16
- Step 2. Combine B and C into BC with a frequency of 22
- Step 3. Combine BC and DE into BCDE with a frequency of 38
- Combine A with BCDE into ABCDE with a frequency of 60



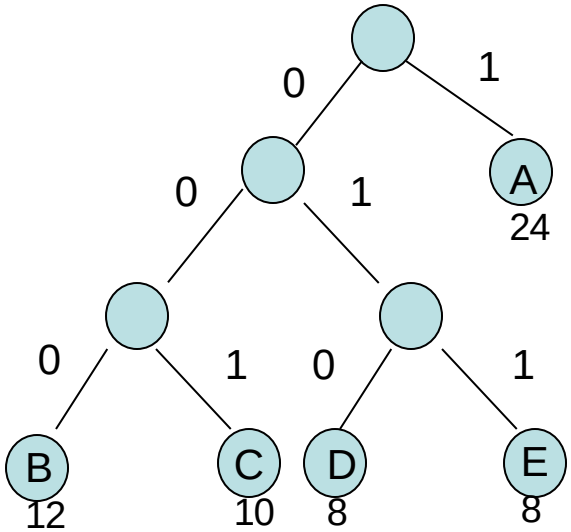
- Building the code tree according to Huffman results into a compressed encoding

Symbol	Frequency	Code	Code Length	Total Length
A	24	1	1	24
B	12	000	3	36
C	10	001	3	30
D	8	010	3	24
E	8	011	3	24

---

186 bit
(3 bit code)

138 bit
(Huffman encoding)



Huffman decoding

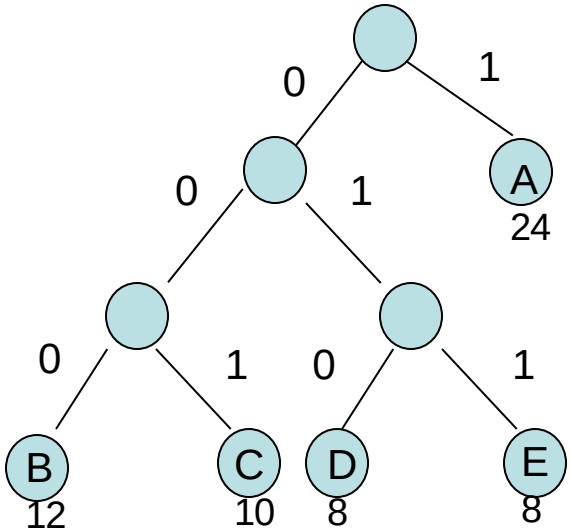
- In order to decode Huffman encoded files, the decoding algorithm must know what code was used to encode the data. A table containing symbols and their codes or the Huffman tree can be used.
- Decoding a file is a two step process. First the header data with the table is read in, and the Huffman code for each symbol is reconstructed. Then the encoded data is read and decoded.

- The fastest method for decoding symbols is to read the encoded file one bit at time and traverse the Huffman tree according to each of the bits until a leaf containing a symbol is reached. When a bit causes a leaf of the tree to be reached, the symbol contained in that leaf is written to the decoded file, and traversal starts again from the root of the tree.

- Example

Input sequence 1**011**000**010**011

Decoded sequence A E B D E



Huffman coding: a simple Python example

```
from collections import Counter
import math

class Node:
    def __init__(self, freq, char=None):
        self.freq = freq
        self.char = char
        self.left = None
        self.right = None

def build_huffman_tree(frequencies):
    nodes = [Node(freq, char) for char, freq in frequencies.items()]
    while len(nodes) > 1:
        nodes.sort(key=lambda x: x.freq)  # sort nodes based on frequency
        # the two nodes with the lowest frequencies are removed from the head of the list
        left = nodes.pop(0)
        right = nodes.pop(0)
        # A new parent node is created (freq. is the sum of the freq. of the two removed nodes)
        parent = Node(left.freq + right.freq)
        parent.left = left
        parent.right = right
        nodes.append(parent)
    # return root of Huffman tree
    return nodes[0]
```

Huffman coding: a simple Python example

```
def generate_codebook(node, code="", codebook={}):
    if node.char is not None:
        codebook[node.char] = code
    else:
        generate_codebook(node.left, code + "0", codebook)
        generate_codebook(node.right, code + "1", codebook)
    return codebook

def obtain_prefix_free_codebook(string):
    frequencies = Counter(string)
    print(frequencies)
    huffman_tree = build_huffman_tree(frequencies)
    codebook = generate_codebook(huffman_tree)
    return codebook
```

- Although Huffman's original algorithm is optimal for a stream of unrelated symbols with a known input probability distribution, it is not optimal when the symbol-by-symbol restriction is dropped, or when the probability mass functions are unknown, …..

- Other methods such as f.e. *LZW coding* used in GIF images often have better compression capability
- These methods can combine an arbitrary number of symbols for more efficient coding, and generally adapt to the actual input statistics when input probabilities are not precisely known or vary significantly within the stream.

- LZW is about frequency of repetitions and Huffman is about frequency of single byte occurrence.
  - Consider the string: 123123123 the frequences of 1, 2 and 3 are the same but we can spot that there are 3 repetitions of 123.

LZW coding

- LZW compression replaces sequences with two or more symbols with single codes. Every new string of symbols LZW finds is added to a table of strings. Compression occurs when a single code is output instead of a string of symbols

- The LZW code can be of any arbitrary length, but it must have more bits in it than a single symbol

- For 8 bit symbols a 12 bit code can be used, where codes 0-255 are assigned to the standard set of symbols and codes 256-4095 refer to strings that are found as the algorithm proceeds

- Used in GIF, TIFF and PDF files.

Example

The input character string is a list of words separated by '/' : /WED/WE/WEE/WEB/WET

- A table with 256 individual characters is assumed to be present. Each character is assigned with a unique code (0-255).
  - Starting from the first *couple of characters*, the string **/W** is checked if in the table. Since it is not present in the table, the string "/W" is added to the table with code 256 (codes 0-255 are already used for 256 characters) and the code for " / " is output.
  - After " E ", has been read in, the string **WE** is checked and then added to the table with code 257, and the code for " W " is output.
  - After " D ", has been read in, the string **ED** is checked and then added to the table with code 258, and the code for " E " is output.
  - After " / ", has been read in, the string **D/** is checked and then added to the table with code 259, and the code for " D " is output. …..
  - Then the characters " W " is read in, the string **/W** is checked. Since it matches string with code 256 in the table, a new character " E " is read in. The string **/WE** (*three characters*) is now checked. Since it is not present in the table , the string "/WE" is added to the table with code 260 and the code 256 (corresponding to “/W”) is output.
  - The process continues until the string is exhausted and all the codes have been output.

/WED /WE /WEE/ WEB/ WET

INPUT Character	existing?	string checked	existing?	OUTPUT Code	Table New code	New String
/	Y	-	-	-	-	-
W	Y	/W	N	/	256	/W
E	Y	WE	N	W	257	WE
D	Y	ED	N	E	258	ED
/	Y	D/	N	D	259	D/
W	Y	/W	Y			
E	Y	/WE	N	256	260	/WE
/	Y	E/	N	E	261	E/
W	Y	/W	Y			
E	Y	/WE	Y			
E	Y	/WEE	N	260	262	/WEE
/	Y	E/	Y			
W	Y	E/W	N	261	263	E/W
E	Y	WE	Y			
B	Y	WEB	N	257	264	WEB
/	Y	B/	N	B	265	B/
W	Y	/W	Y			
E	Y	/WE	Y			
T	Y	/WET	N	260	266	/WET
EOF				T		

LZW coding: a simple Python example

```
def lzw_encode(message):
    # init dictionary for the first 256 entries (ASCII chars)
    dictionary_size = 256
    dictionary = {chr(i): i for i in range(dictionary_size)}
    # encoding ...
    encoded_message = []
    sequence = ""  # current sequence of characters encountered during encoding
    # for each character in the message
    for char in message:
        # if already in dictionary just continue to add characters
        if sequence + char in dictionary:
            sequence += char
        else:  # if the concatenated sequence does not exist in the dictionary, add it
            encoded_message.append(dictionary[sequence])  # outputs previous sequence
            dictionary[sequence + char] = dictionary_size  # add new code for new sequence
            dictionary_size += 1
            sequence = char  # start the sequence from the last char
    encoded_message.append(dictionary[sequence])
    return encoded_message
```

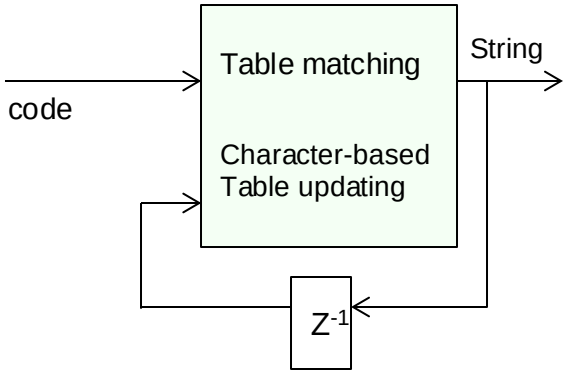
LZW decoding

- The decompression algorithm takes the stream of codes output from the compression algorithm and uses them to recreate the input stream.
- The LZW algorithm does not need to pass the string table to the decompression code. The table can be built exactly as it was during compression, using the input stream

Input Codes: / W E D 256 E 260 261 257 B 260 T

INPUT		Output Character	String checked		New table entry
/	Y	/	/		
W	Y	W	/W	N	256 = /W
E	Y	E	WE	N	257 = WE
D	Y	D	ED	N	258 = ED
256	Y	/W	D/	N	259 = D/
E	Y	E	/WE	N	260 = /WE
260	Y	/WE	E/	N	261 = E/
261	Y	E/	/WEE	N	262 = /WEE
257	Y	WE	E/W	N	263 = E/W
B	Y	B	WEB	N	264 = WEB
260	Y	/WE	B/	N	265 = B/
T	Y	T	/WET	N	266 = /WET

A table with 256 individual characters is assumed to be present.



LZW decoding: a simple Python example

```
def lzw_decode(encoded_message):
    dictionary_size = 256
    dictionary = {i: chr(i) for i in range(dictionary_size)}
    decoded_message = ""
    sequence = chr(encoded_message.pop(0))
    decoded_message += sequence
    # for each code in the encoded message
    for code in encoded_message:
        entry = ""
        if code in dictionary:
            entry = dictionary[code]
        elif code == dictionary_size:  # if the code does not exist in the dictionary
            entry = sequence + sequence[0]
        else:
            raise ValueError("Invalid code")
        decoded_message += entry
        dictionary[dictionary_size] = sequence + entry[0]
        dictionary_size += 1
        sequence = entry
    return decoded_message
```

GIF image representation standard

- *GIF Graphic Interchange Format* is an 8 bit per pixel image format using a palette of up to 256 distinct colors from the 24-bit RGB color space. A GIF image employs *lossless LZW compression*. Its lossless compression preserves very sharp edges.

  Supported by: Chrome, Edge, Firefox, IE, Opera, Safari

- The 256 colors limitation makes the GIF format unsuitable for color photographs and other images with continuous color. It is instead well-suited for simpler images such as graphics, sharp-edged line art or logos with solid areas of color with a very limited number of colors.

- Monochrome photographs (with continuous grey tones) can be represented well as GIFs but have large file sizes due to the inappropriate compression technique

- Formerly widely used on the web, PNG has increasingly replaced GIFs on web pages, but services like Faceboom and instagram have added support for it (e.g. for stickers)…

GIF animation

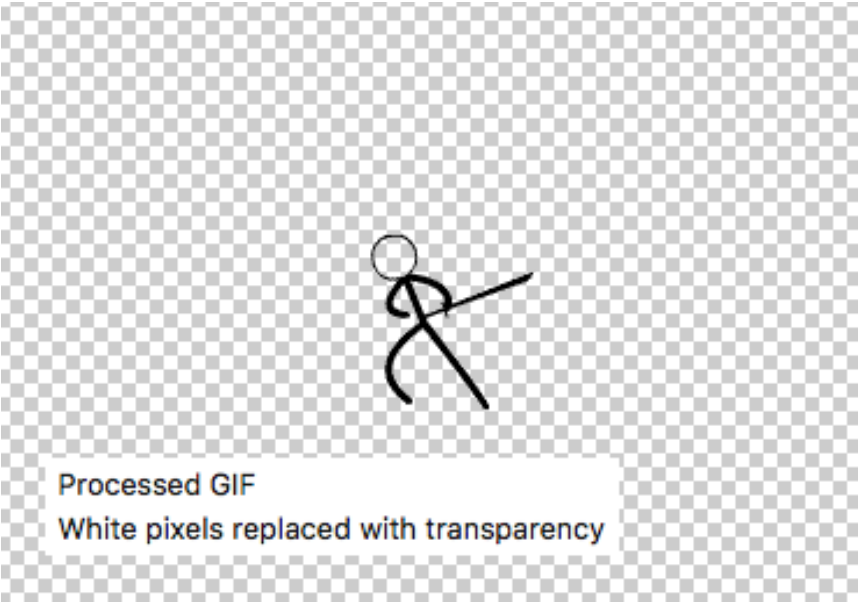
- GIF supports simple animation, in which following an initial full-size frame, a series of images reflecting the parts of the image that change with each frame are provided. An internal clock embedded into the GIF keeps count and flips the image when the time comes.

- It’s possible to have a different palette for each frame of the animation.

GIF Transparency

- With GIF, pixels are opaque, unless a specific color index is designated as transparent, in which case pixels colored that value are entirely transparent. A transparent GIF is limited in that only one color of the 256-shade palette can be made transparent

- The process is similar to *chroma key* used in television. A computer is told to hone in on a specific color, let's say it's green. That chroma is then erased and replaced by another image.



PNG image representation standard

- *PNG Portable Network Graphics* is a lossless compression format created to improve and replace the GIF format. It overcomes the GIF limitation to 256 colors.
  PNG files use file-extension "PNG" or "png"
  Supported by: Chrome, Edge, Firefox, IE, Opera, Safari.

- PNG was designed for transfering images on the internet avoiding legal aspects of LZW patent used in GIF. It is the optimal choice for exporting images with repeating gradients for web usage (24-bit RGB palettes).

- PNG did not support EXIF (Exchangeable Image File) image data (including camera settings like shutter speed, focal lenght, exposure compensation, flash used… and scene information, date and time…) which makes it problematic for use of professional photographers.
  - PNG supports chunks. Standardized EXIF chucks has been added later (since July 2017).

- PNG is useful for saving temporary photographs that require successive editing. When the photograph is ready to be distributed, it can then be saved as a JPEG (later), and this limits the information loss to just one generation.

DEFLATE compression

- PNG employs the DEFLATE lossless compression algorithm (also used in the PKZIP archiving tool and specified in RFC 1951). A DEFLATE stream consists of a series of blocks. Each block uses a single mode of compression and is preceded by a header.

- DEFLATE compression is achieved through two steps:
  - *LZ77 compression algorithm**: matching and replacement of duplicate strings with pointers: if a repeated string of bits exists a back reference to the previous location of the same string is inserted.
  - *Huffman coding*: replacing symbols with new, weighted symbols based on frequency of use

  - *actually LZSS variant

The LZ77 compression

- *The LZ77 compression* finds sequences of characters that are repeated. It uses a sliding window (search buffer) of 32K (i.e. records what the last 32768 characters were).

- When the same sequence of characters is encountered the sequence is replaced by:
  - ***distance*** (how far back in the window) – 15-bit distance = 32.768 bytes
  - ***lenght*** (the number of characters to check if identical) – 8-bit (3-258 repeated characters),

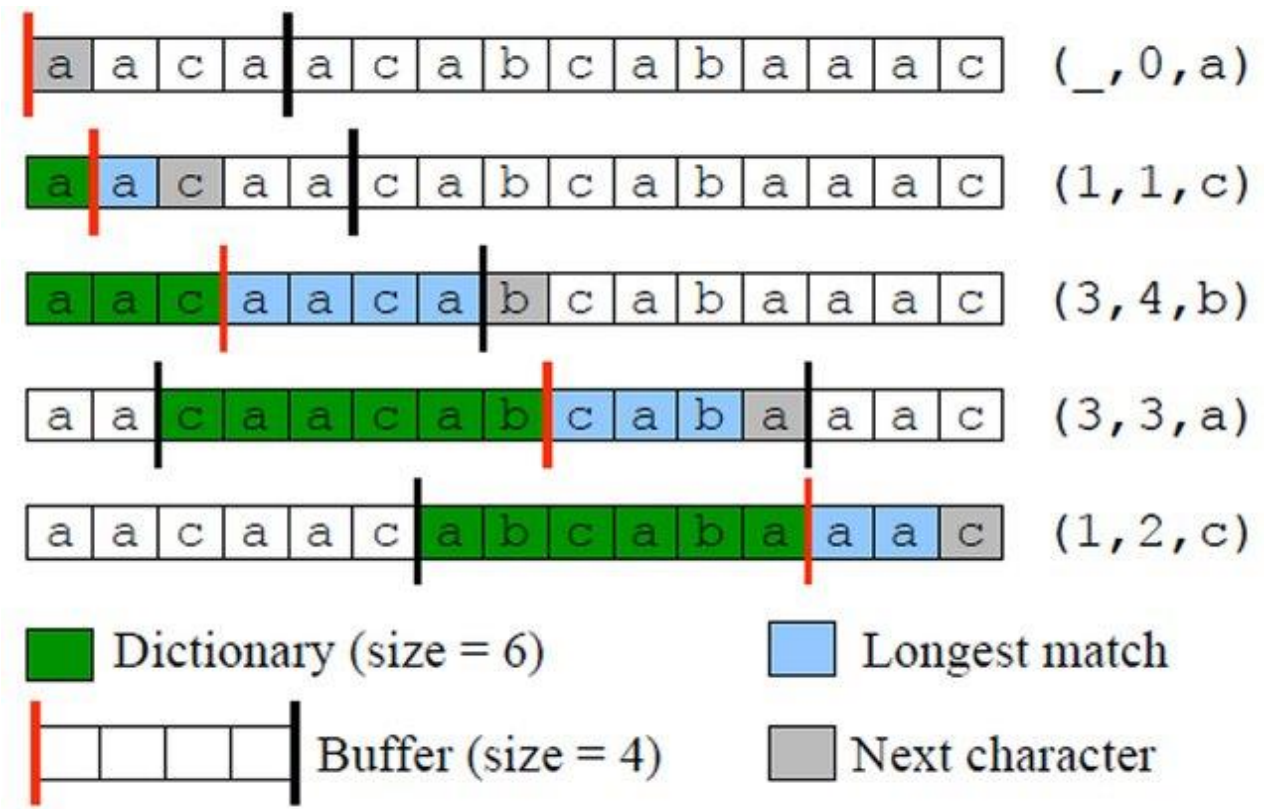
  which is equivalent to the statement: "*each of the next* [**length]** *characters is equal to the characters exactly* **[distance]** *characters behind it in the uncompressed stream*". It' like a copy command. Then moves to ***lenght+1*** position.

- It is acceptable (and frequently useful) to allow length-distance pairs to specify a length that exceeds the distance. Tackling one byte at a time, as a byte is copied over, it may be fed again as input to the copy command: the operation is thus equivalent to the statement "*copy the data you were given and repetitively paste it until it fits*".

- *LZSS* improves on LZ77 by using a 1-bit flag to indicate whether the next chunk of data is a literal or a length–distance pair and using literals if a length–distance pair would be longer.

Example

Dictionary and buffer windows slide with the cursor. The output is a triplet (p, l, c) where:
P = position of the longest match that starts in the dictionary (relative to the cursor)
l = length of longest match
C = next char in the buffer beyond longest match
Advance the window by l+1 positions



Elias gamma coding

- Universal code that doesn't require a dictionary/tree
  - No need to store dictionary information as Huffman
- Works well when you don't know the distribution of values in advance
  - It’s not as optimal as Huffman coding
- Simple to implement and fast to encode/decode
  - This is the real deal !
- Very efficient for small integers (which LZ77 or RLE often produce)

- Elias gamma coding encodes a positive integer n in two parts:
  - Unary part: Represents the length of the binary representation of n minus 1 as a series of zeros
  - Binary part: The binary representation of n with the leading 1 included

- Encoding Process:
  - Convert the number n to its binary representation
  - Count the number of bits in this binary representation (let's call this L)
  - Write L-1 zeros (this is the unary part)
  - Write the binary representation of n (this is the binary part)

Elias gamma coding

- Encoding Process:
  - Convert the number n to its binary representation
  - Count the number of bits in this binary representation (let's call this L)
  - Write L-1 zeros (this is the unary part)
  - Write the binary representation of n (this is the binary part)

- Decoding Process:
  - Count the number of leading zeros (let's call this k)
  - Read the next k+1 bits and interpret as a binary number
  - This gives you the original number n

- Example:
  - For n = 5:
    - Binary representation: 101
    - Length (L) = 3
    - Unary part: 2 zeros (since L-1 = 2)
    - Binary part: 101
    - Encoded: 00101

Elias gamma coding and LZ77 triplet

- For triplet (3, 2, 'A'):

- Encode distance (3):
  - Add 1: 3 + 1 = 4
  - Binary: 100
  - Length: 3
  - Elias gamma: 00100

- Encode length (2):
  - Add 1: 2 + 1 = 3
  - Binary: 11
  - Length: 2
  - Elias gamma: 011

Elias gamma coding

```
def encode_number(n):
    """Encode a positive integer using Elias gamma coding"""
    if n == 0:
        return "0"

    # Find the largest power of 2 less than or equal to n
    binary = bin(n)[2:]  # Convert to binary and remove '0b' prefix
    length = len(binary)

    # Elias gamma consists of (length-1) zeros followed by the binary representation
    return '0' * (length - 1) + binary
```

Elias gamma decoding

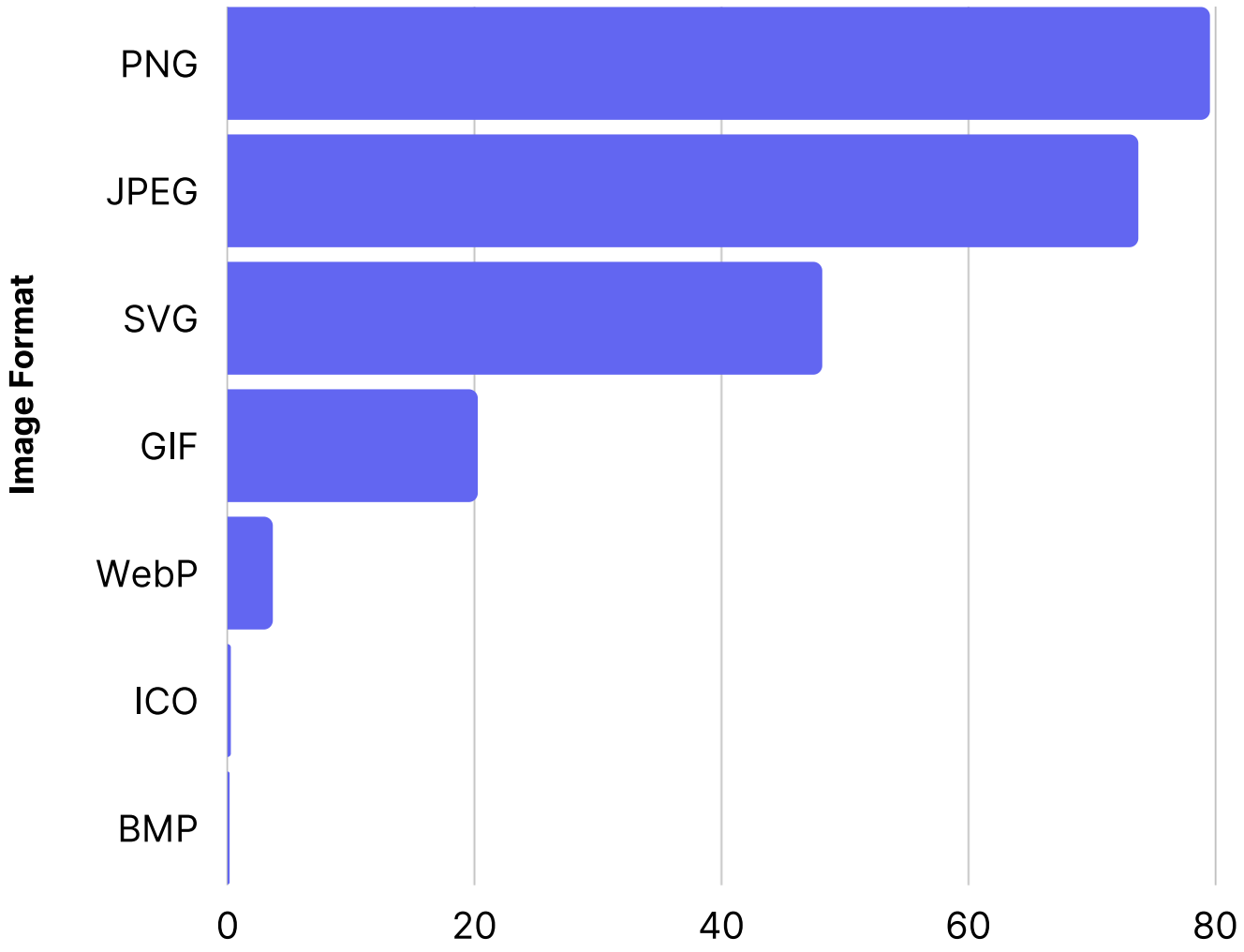
```
def decode_number(bitstring, start_pos=0):
    """
    Decode an Elias gamma coded number from a bitstring.
    Returns tuple (decoded_number, new_position).
    """
    if bitstring[start_pos] == '0':
        # Count the number of leading zeros
        num_zeros = 0
        pos = start_pos
        while pos < len(bitstring) and bitstring[pos] == '0':
            num_zeros += 1
            pos += 1

        if pos >= len(bitstring):
            raise ValueError("Unexpected end of bitstring")

        # Read num_zeros + 1 bits (including the first '1')
        binary = bitstring[pos:pos + num_zeros + 1]
        if len(binary) < num_zeros + 1:
            raise ValueError("Unexpected end of bitstring")

        decoded_number = int(binary, 2)
        return decoded_number, pos + num_zeros + 1
    else:  # Starts with '1', so number is 1
        return 1, start_pos + 1
```

- PNG supports grayscale or RGB images.
  The number of channels depends on whether the image is greyscale or color and whether it has an alpha channel. Basically, through the alpha channel, a clip can specify its degree of opacity and facilitate all the superimposing operations in which it is involved.

- PNG allows the following combinations of channels:
  - greyscale
  - greyscale and alpha (0/1 indicates the level of transparency for each pixel)
  - red, green and blue (RGB / truecolor)
  - red, green, blue and alpha

Type	cell values are total bits per pixel				
greyscale	1	2	4	8	16
greyscale & alpha	No	No	No	16	32
Truecolor	No	No	No	24	48
Truecolor & alpha	No	No	No	32	64

Image Format Usage on Websites



Data Source: W3 Techs. Data as at 5th April 2022

% of Websites using image format on website